
DSC5001: Statistical Machine Learning I

Modern Machine Learning and Project Directions

Kangning Cui

City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning Objectives

By the end of this lecture, you should be able to:

1. connect classical statistical learning ideas to modern ML systems;
2. recognize major modern model families and their typical data types;
3. design a realistic project question with baselines and evaluation;
4. avoid common project failures such as leakage, weak comparison, and unclear metrics.

Modern machine learning is not only about large models; it is about choosing the right model, data, and evidence for the question.

From statistical learning to modern machine learning

DSC5001 ■ Semester A, 2026/27

The Core Question Has Not Changed

In this course, many models are used to estimate or approximate:

$$\mathbb{E}[Y \mid X = x].$$

Modern ML often uses larger models and richer data, but the basic question remains:

How well can we learn a useful mapping from input to output?

The input may be:

table, image, text, signal, graph, multiple modalities.

The model becomes more complex, but the need for careful evaluation becomes even stronger.

What Changes in Modern ML?

Compared with simple regression models, modern ML often changes three things.

Representation	The model learns features rather than relying only on hand-designed variables.
Scale	Training may use large datasets, pretrained models, and many parameters.
Reuse	A model trained for one setting may be adapted to another setting.

Main idea

Modern ML extends the same statistical learning problem to richer representations and larger systems.

Classical Pipeline and Modern Pipeline

	Classical statistical ML	Modern ML
Features	often manually designed	often learned from data
Model size	small to moderate	can be very large
Training data	task-specific	task-specific or large-scale pretraining
Main risk	wrong assumptions, overfitting	overfitting, leakage, compute, hidden bias
Evaluation	validation and test data	validation, test data, stress tests, error analysis

The vocabulary changes, but the statistical discipline remains essential.

A Useful Mental Model

A machine-learning project contains four linked parts:

data → representation → model → evaluation.

A weak project often starts with:

“I want to use a Transformer.”

A stronger project starts with:

“What prediction problem do I need to solve, and how will I know if it improves?”

Common mistake

A model name is not a research question.

Major model families

DSC5001 ■ Semester A, 2026/27

Data Structure Guides Model Choice

Data type	Useful structure	Common model choices
Tabular	mixed numerical and categorical features	linear models, trees, boosting
Images	local spatial patterns	CNNs, vision transformers
Text	sequence and context	transformers, pretrained language models
Time series	order and temporal dependence	filters, RNNs, transformers
Graphs	relationships among entities	graph neural networks
Multiple modalities	alignment across data types	contrastive and fusion models

A good model uses structure that is actually present in the data.

Strong Baselines Still Matter

Before using a complex model, build a simple baseline.

Examples:

- ▶ mean or majority-class predictor;
- ▶ linear or logistic regression;
- ▶ random forest or gradient boosting;
- ▶ simple neural network;
- ▶ pretrained features plus a linear classifier.

A baseline answers:

How much improvement does the complex method really provide?

Common mistake

A complicated model is not convincing unless it beats appropriate simpler alternatives.

Trees and Boosting

Decision trees split the feature space using rules such as

$$x_j < c.$$

They are useful for tabular data because they can capture:

- ▶ nonlinear effects;
- ▶ feature interactions;
- ▶ mixed feature types.

Boosting builds an ensemble sequentially:

new tree focuses on current mistakes.

For many tabular projects, boosted trees are a strong non-neural baseline.

Neural Networks

A neural-network layer has the form

$$\mathbf{h}^{(\ell+1)} = \phi \left(\mathbf{W}^{(\ell)} \mathbf{h}^{(\ell)} + \mathbf{b}^{(\ell)} \right).$$

Stacking layers gives a flexible nonlinear function.

The model learns:

representation + prediction rule.

Neural networks are useful when the input structure is complex, but they usually require:

- ▶ enough data;
- ▶ regularization;
- ▶ careful validation;
- ▶ meaningful baselines.

Convolutional Neural Networks

A convolution applies the same local filter across positions:

$$y[t] = \sum_s w[s]x[t - s].$$

This gives two important properties:

- ▶ **locality:** nearby pixels or samples are processed together;
- ▶ **parameter sharing:** the same filter is reused across locations.

Typical uses include:

images, signals, volumes, video.

Main idea

CNNs are useful when local patterns repeat across the input.

Attention and Transformers

Attention lets one item use information from other items.
The standard form is:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{V}.$$

The roles are:

- ▶ **query:** what information is needed;
- ▶ **key:** what each item offers for matching;
- ▶ **value:** what information is combined.

Transformers use attention to build context-dependent representations.

Pretraining and Transfer Learning

Pretraining learns reusable parameters from a large source dataset.

A target task can then use the pretrained model by:

- ▶ freezing the representation and training a new head;
- ▶ fine-tuning the whole model;
- ▶ updating a small adapter;
- ▶ using prompts when appropriate.

For a course project:

adapting an existing model is usually more realistic than training a large model from scratch.

Generative Models

Generative models describe how data may be produced.

Examples include:

- ▶ **autoregressive models:** predict the next element;
- ▶ **latent-variable models:** explain data using hidden variables;
- ▶ **diffusion models:** learn to reverse a noising process.

For example, an autoregressive model writes:

$$p(x) = \prod_t p(x_t \mid x_{<t}).$$

Generative models are useful for synthesis, representation learning, simulation, and data understanding.

Foundation Models

A foundation model is pretrained broadly and then adapted to many tasks.

Examples include:

language models, vision encoders, vision-language models.

For projects, the key question is not:

Can we use a foundation model?

The better question is:

Does it improve the target task under a fair comparison?

Designing project evidence

DSC5001 ■ Semester A, 2026/27

From Topic to Testable Question

Too broad:

“Use deep learning for medical images.”

Better:

“Does transfer learning improve classification when labels are limited?”

A testable project question should specify:

- ▶ the task;
- ▶ the dataset;
- ▶ the comparison;
- ▶ the performance measure;
- ▶ the main constraint.

A narrow question is easier to evaluate than a broad application area.

Project Question Template

A useful project question often has this form:

Does method A improve metric M over baseline B on dataset D under constraint C?

Examples:

- ▶ Does a pretrained image encoder improve accuracy under limited labels?
- ▶ Do boosted trees outperform logistic regression on tabular risk prediction?
- ▶ Does adding text improve prediction beyond structured variables?
- ▶ Does a spline or nonlinear model improve over a linear baseline?

The question should imply what evidence is needed.

Controlled Comparison Ladder

A convincing project usually compares multiple levels of complexity:

1. trivial reference model;
2. simple classical model;
3. strong standard baseline;
4. proposed model or adaptation.

Example:

majority class $\rightarrow$ logistic regression $\rightarrow$ boosted trees $\rightarrow$ pretrained representation.

Main idea

One controlled change gives one interpretable conclusion.

Data Splitting and Leakage

Before training, identify the independent sample unit.

Examples:

- ▶ patient;
- ▶ user;
- ▶ video;
- ▶ location;
- ▶ time period;
- ▶ document.

The split should prevent related samples from appearing in both train and test sets.

Common mistake

If frames from the same video or records from the same patient appear in both training and testing, performance may be overly optimistic.

Evaluation Metrics

Choose the metric according to the project goal.

Goal	Possible metrics
balanced classification	accuracy, macro-F1
rare event detection	sensitivity, precision, recall
numerical prediction	MSE, MAE
segmentation	Dice, IoU, boundary error
ranking or retrieval	top- k accuracy, relevance score
probability prediction	log loss, calibration error

A good metric reflects the decision the model is meant to support.

Validation Is for Choosing

Validation data are used to choose:

model family, hyperparameters, preprocessing, thresholds.

The test set should be used only after model selection.

train → validate → select → test once.

If the test set guides modeling decisions, it becomes part of the training process.

Error Analysis

Average scores are not enough.

Inspect:

- ▶ class-specific errors;
- ▶ high-confidence mistakes;
- ▶ examples where methods disagree;
- ▶ performance across subgroups;
- ▶ sensitivity to preprocessing.

The purpose is to answer:

Where does the model fail, and why?

This often produces the most useful part of a project discussion.

Reproducibility Checklist

A project result should be traceable.

Record:

- ▶ dataset source and version;
- ▶ train, validation, and test split;
- ▶ preprocessing;
- ▶ model configuration;
- ▶ random seed;
- ▶ software environment;
- ▶ evaluation code and metric definition.

A reported number is meaningful only when the configuration that produced it is clear.

Project directions

DSC5001 ■ Semester A, 2026/27

Direction 1: Tabular Prediction

Possible tasks:

- ▶ risk prediction;
- ▶ customer behavior analysis;
- ▶ housing or price prediction;
- ▶ medical outcome prediction from structured variables.

Recommended comparison:

linear model → tree or boosting → feature engineering or calibration.

Good project angle:

When does nonlinear structure improve prediction?

Direction 2: Image Classification

Possible tasks:

- ▶ medical image classification;
- ▶ remote-sensing scene classification;
- ▶ defect detection;
- ▶ plant or object recognition.

Recommended comparison:

small CNN → pretrained encoder → fine-tuning or feature extraction.

Good project angle:

How much does transfer learning help under limited labels?

Direction 3: Text or Language Data

Possible tasks:

- ▶ sentiment classification;
- ▶ topic classification;
- ▶ fake review detection;
- ▶ question or document categorization.

Recommended comparison:

bag-of-words $\rightarrow$ TF-IDF + linear model $\rightarrow$ pretrained embeddings or language model.

Good project angle:

Does a pretrained representation improve over simple text features?

Direction 4: Time Series and Signals

Possible tasks:

- ▶ anomaly detection;
- ▶ activity recognition;
- ▶ sensor classification;
- ▶ short-term forecasting.

Recommended comparison:

summary features → classical ML → sequence or neural model.

Good project angle:

Which temporal information is actually useful for prediction?

Direction 5: Multimodal Data

A multimodal project uses more than one data source, such as:

image + text

or

structured variables + signals.

The central question should be:

Does the additional modality improve prediction beyond the strongest single-modality baseline?

A weak comparison only shows that multimodal models work.

A stronger comparison shows what the extra modality adds.

Project Scope: What Is Realistic?

For this course, a strong project does not need to train a large model from scratch. Realistic options include:

- ▶ compare classical baselines carefully;
- ▶ use pretrained features;
- ▶ fine-tune a small model;
- ▶ run controlled ablations;
- ▶ perform careful error analysis.

Common mistake

An unfinished large project is weaker than a completed controlled experiment.

Before You Start Implementation

Each group should be able to state:

1. **Question:** What claim are you testing?
2. **Data:** What is one independent sample?
3. **Split:** How do you avoid leakage?
4. **Baselines:** What simple models will you compare?
5. **Metric:** What number answers the question?
6. **Failure analysis:** What examples will you inspect?

If any item is unclear, solve it before adding model complexity.

Summary

Three principles organize modern ML project design:

1. **Match structure:** choose methods that fit the data type and learning signal.
2. **Control comparisons:** compare against meaningful baselines.
3. **Design for evidence:** use validation, testing, and error analysis correctly.

sharp question + controlled experiment > oversized model

Modern ML: choose the question before choosing the model

Questions and discussion